

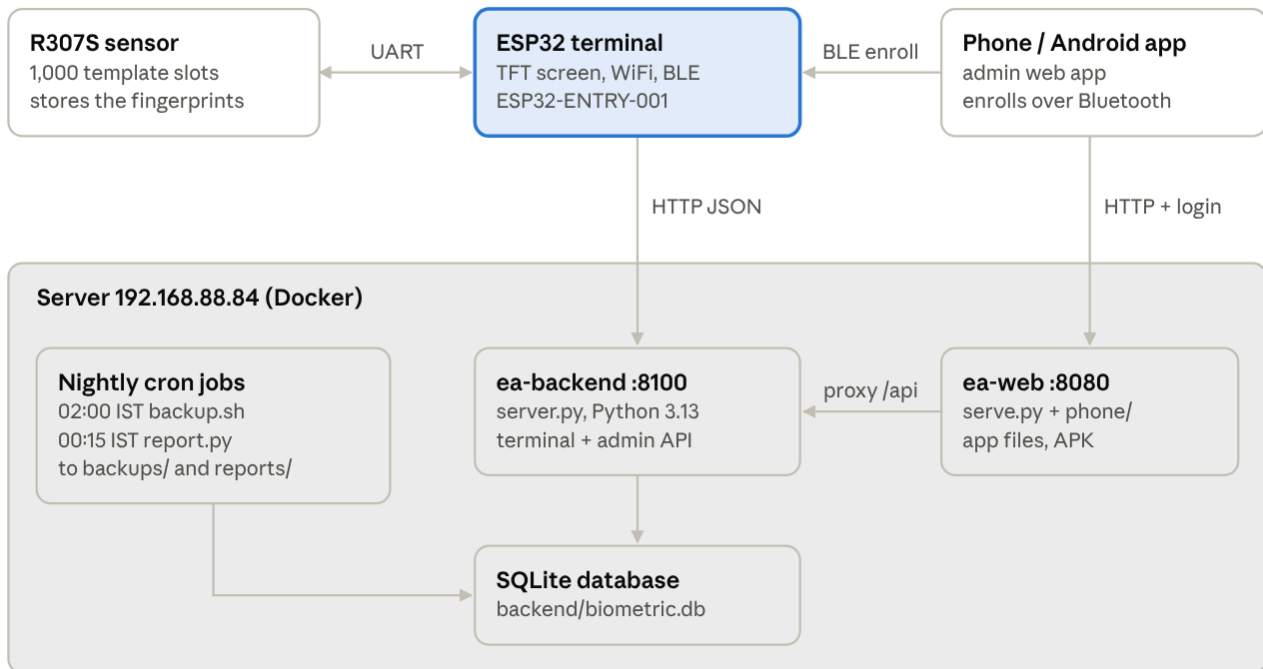
Employees Access — Fingerprint System: Complete Project Guide

Overview

Employees Access is a fingerprint attendance and access system: an ESP32 terminal with an R307S fingerprint sensor matches a finger to a sensor slot, and a Python backend turns that slot into an employee punch (IN or OUT). Admins manage employees, holidays and reports from a phone web app or an Android app. Everything runs on one server, `server@192.168.88.84`, in Docker.

Part	Role	Lives in
Terminal firmware	Reads fingers, enrolls new ones, reports punches over WiFi (BLE also used)	<code>firmware/</code>
Backend API	Employees, slots, punches, attendance, exports, admin login; SQLite	<code>backend/server.py</code>
Web app	Admin UI for phones and browsers, plus a data sheet	<code>phone/</code>
Android app	Installable app that wraps the web app	<code>android/</code>
Deployment	Docker Compose, install, backup and report scripts	<code>deploy/</code>
Outputs	Nightly DB backups and attendance CSV reports	<code>backups/</code> , <code>reports/</code>

The terminal and phones both talk to one server



System architecture · terminal, phone, server

The terminal reports scans to **ea-backend** on port 8100. Phones use **ea-web** on port 8080, which forwards API calls to the backend, and talk to the terminal over Bluetooth only during enrollment.

Source tree

The repo root is `~/fingerprint_system` on the server. Build output (`build/`, `__pycache__/`) and the 1.8 GB ESP-IDF toolchain are left out below. `docs/`, `scripts/` and `test/` exist but are empty.

```

fingerprint_system/
├─ README.md                one-line component list
├─ fingerprint_system.iml, .idea/ IntelliJ / Android Studio project files
├─ firmware/                ESP32 terminal (ESP-IDF projects)
│   └─ hello_world/         ← the real terminal firmware
│       └─ main/
│           └─ hello_world_main.c  main loop: sensor, WiFi, server calls,
display
│   └─ ble_enroll.c/.h      BLE enrollment link to the phone/Android
app
│       └─ json_util.c/.h    small JSON helpers for API calls
│           └─ *.c.before_* / *.backup  older snapshots of main.c
│       └─ components/tft/    TFT display driver, fonts, gen_fonts.py
│       └─ CMakeLists.txt, sdkconfig, sdkconfig.ci, build.txt
  
```

```

| | └─ pytest_hello_world.py  ESP-IDF pytest smoke test
| └─ tft_test/                standalone display test (tft_test.c,
tft_test_colors.c)
└─ backend/
| └─ server.py                the whole API server (Python stdlib, ~1,330
lines)
| └─ biometric.db             LIVE SQLite database
| └─ biometric-backup-*.db    manual backups from 2026-10-05
└─ phone/                    admin web app (PWA)
| └─ index.html, app.js, styles.css  main admin UI
| └─ sheet.html, sheet.js, sheet.css  data sheet view
| └─ android-bridge.js        talks to the Android app's native BLE
bridge
| └─ sw.js, manifest.webmanifest, icons/  offline + install support
| └─ serve.py                 static server + proxy for the web app
| └─ download/employees-access.apk  APK offered for download
└─ android/                  Android wrapper app
| └─ AndroidManifest.xml, res/
| └─ src/com/employeesaccess/admin/MainActivity.java, BleBridge.java
| └─ build.sh                 builds + signs the APK without Gradle
| └─ release.keystore         signing key – keep private
| └─ build/employees-access.apk
└─ deploy/
| └─ docker-compose.yml       ea-backend + ea-web services
| └─ install.sh               start services + install the cron schedule
| └─ backup.sh                nightly DB backup (30-day retention)
| └─ report.py                nightly attendance CSV reports
| └─ README.md                operations guide
└─ backups/                  biometric-YYYYMMDD-HHMM.db, backup.log,
pre-ble-fix tarball
└─ reports/
| └─ daily/                   attendance-DATE.csv, punch-times-DATE.csv
| └─ monthly/                 attendance-YYYY-MM.csv, punch-times-YYYY-
MM.csv
| └─ report.log
└─ tools/esp/esp-idf/        ESP-IDF SDK + toolchain (1.8 GB)

```

Apps, tools and technologies

Layer	Tool / technology	Used for
Firmware	ESP-IDF v6.1-dev (<code>tools/esp/esp-idf</code>), <code>idf.py</code> , xtensa-esp32-elf GCC	Build and flash the ESP32

Layer	Tool / technology	Used for
Firmware	FreeRTOS, NimBLE, <code>esp_http_client</code> , <code>esp_wifi</code> , NVS	Tasks, Bluetooth LE, HTTP calls, WiFi, storage
Hardware	ESP32, R307S fingerprint sensor, 240×320 SPI TFT	The terminal
Backend	Python 3.13 standard library, SQLite 3	API server and database
Web	HTML, CSS, vanilla JavaScript, service worker (PWA), Web Bluetooth	Admin app
Android	Java, Android SDK build tools (<code>aapt2</code> , <code>d8</code> , <code>zipalign</code> , <code>apksigner</code>), Android Studio JDK	APK wrapper with native BLE
Server	Ubuntu Linux, Docker + Docker Compose (<code>python:3.13-slim</code> image), cron	Hosting, auto-restart, nightly jobs
Dev	IntelliJ IDEA / Android Studio (<code>.idea/</code> , <code>fingerprint_system.iml</code>), Git	Editing and version control
Office tools	Any spreadsheet app (Excel, LibreOffice, Google Sheets)	Opening CSV exports and reports

Firmware (ESP32 terminal)

The terminal firmware is the ESP-IDF project `firmware/hello_world` (despite its name). It is about 2,300 lines of C, version `0.1.0` , built with ESP-IDF `v6.1-dev` for a classic ESP32 with 4 MB of flash. `firmware/tft_test` is only a display test.

Hardware and pins

Part	Interface	Pins / settings
R307S fingerprint sensor (1,000 slots; slot 0 unused)	UART2	TX GPIO17, RX GPIO16, 57600 baud
TFT display, 240 × 320	SPI2	SCLK 18, MOSI 23, MISO 19, CS 5, DC 21, RESET 22
WiFi	station mode	SSID <code>Ramdisk</code> (password in source)
Bluetooth LE	NimBLE	advertised as <code>CBIO-ENTRY-001</code>

Compile-time settings (`main/hello_world_main.c`)

Constant	Value
<code>SERVER_BASE_URL</code>	<code>http://192.168.88.84:8100</code>
<code>DEVICE_ID</code>	<code>ESP32-ENTRY-001</code>
<code>FIRMWARE_VERSION</code>	<code>0.1.0</code>
<code>TIMEZONE</code> / <code>NTP_SERVER</code>	<code>IST-5:30</code> / <code>pool.ntp.org</code>
<code>WIFI_SSID</code> / <code>WIFI_PASSWORD</code>	hard-coded; change here and reflash

Source files

- `main/hello_world_main.c` : boot, WiFi, NTP, the R307S driver (search, capture, store), HTTP calls to the backend and the main verify loop.
- `main/ble_enroll.c/.h` : the BLE GATT enrollment service and passkey pairing.
- `main/json_util.c/.h` : small JSON read and escape helpers.
- `components/tft/` : the display driver, its screens (startup, place finger, granted, denied, punch, passkey) and `tools/gen_fonts.py` .

Build and flash

The ESP32 must be plugged into the machine that flashes it.

```
. ~/fingerprint_system/tools/esp/esp-idf/export.sh    # load ESP-IDF into the
shell
cd ~/fingerprint_system/firmware/hello_world
idf.py set-target esp32          # first time only
idf.py build
idf.py -p /dev/ttyUSB0 flash monitor    # Ctrl+] leaves the monitor
```

The boot log prints the device ID, firmware version, server URL, BLE name and sensor UART settings. `build.txt` holds an old failed build log (a `GPIO_NUM_17` error from an earlier version).

Backend server and protocols

The backend is one file, `backend/server.py` . It uses only the Python standard library (`http.server` with a `ThreadingHTTPServer` , plus `sqlite3`) and runs on Python 3.13 in Docker. All requests are handled one at a time under a lock, so slot assignment cannot race.

```
python3 server.py # 0.0.0.0:8000, ./biometric.db
python3 server.py --port 8100 --db /path/to/biometric.db
python3 server.py --set-password # set admin password, log everyone
out
```

Protocol summary

Link	Transport	Format	Security
Terminal → backend	HTTP on port 8100 (LAN)	JSON POST	none (open endpoints)
Browser/app → web server	HTTP on port 8080	static files + JSON proxied to 8100	admin session cookie
Phone → terminal (enrollment)	Bluetooth LE GATT	JSON on characteristics	passkey pairing (MITM, bonded)
Sensor → ESP32	UART2, 57600 baud	R307S packet protocol	sensor password check
ESP32 clock	NTP pool.ntp.org	—	—

Responses are JSON. Errors look like `{"detail": "..."} with a 4xx status: 404 for not found, 409 for a conflict (duplicate code, slot taken, memory full, inactive employee) and 422 for bad input.`

Terminal endpoints (no login)

Method + path	Request body	Response
POST /api/v1/devices/heartbeat	device_id, firmware_version	{status:"ok", device_id, company_id}
POST /api/v1/enrollment/start	employee_id, device_id, finger_index (1-10)	the free sensor slot to store into
POST /api/v1/enrollment/complete	employee_id, device_id, finger_index, sensor_slot (1-999)	201 {status:"enrolled", sensor_slot, full_name}

Method + path	Request body	Response
POST /api/v1/access-events	device_id, sensor_slot, match_score	{status: allowed/denied, reason, punch: in/out, duplicate, detail, worked, full_name}
GET /health, GET /health/database	—	{status:"ok"}

Denial reasons are `unknown_sensor_slot`, `employee_inactive` and `biometric_record_inactive`. A second scan within 30 seconds returns `reason: "repeat_scan"` and records no new punch.

Admin endpoints (login required)

Area	Endpoints
Login	POST /api/v1/auth/login {password} → cookie <code>ea_session</code> (12 h, HttpOnly, SameSite=Strict); POST /api/v1/auth/logout; GET /api/v1/auth/me
Employees	GET /api/v1/employees; POST /api/v1/employees {company_id, employee_code, full_name, department?, email?, phone?}; POST /api/v1/employees/delete {employee_id}
Logs	GET /api/v1/logs?type=&from=&to=&limit=
Attendance	GET /api/v1/attendance?date=YYYY-MM-DD or ?from=&to=
Calendar	GET /api/v1/calendar; POST /api/v1/calendar/weekend {weekend_days: [5,6]} (Monday = 0); POST /api/v1/holidays {date, name}; POST /api/v1/holidays/delete {date}
CSV exports	GET /api/v1/export/logs.csv, employees.csv, attendance.csv?from=&to=, punches.csv?from=&to=

Area	Endpoints
Saved reports	GET /api/v1/reports ; GET /api/v1/reports/download?name=daily/attendance-2026-10-05.csv
Live updates	GET /api/v1/changes?since=N&wait=8 : long-poll that answers as soon as data changes (8 s max)

The admin password is stored as a PBKDF2-SHA256 hash (240,000 rounds, minimum 8 characters). After 5 wrong attempts from one IP address, login is locked for 60 seconds.

BLE enrollment service (terminal side)

Item	UUID / value
Service	6e1a0001-4b5c-4c1e-9f3a-2d7c5e8b9a01
Command (write)	6e1a0002-... — phone writes {"employee_id":5,"finger_index":1}
Status (read/notify)	6e1a0003-... — terminal sends {"state":...}
Pairing	6-digit passkey shown on the TFT; bonding + MITM protection

The status `state` values in order are `idle`, `queued`, `contacting_server`, `place_finger` (step 1), `remove_finger`, `place_finger` (step 2), `saving`, then `done` `{slot, name}`. Any failure sends `error` `{message}`.

Database schema (SQLite)

Table	Holds
<code>companies</code>	one default company (id 1)
<code>employees</code>	code, name, department, email, phone, active, <code>deleted_at</code> (deleted employees are kept so old logs still show names)
<code>devices</code>	terminal ID, firmware version, last seen
<code>biometric_records</code>	employee ↔ terminal ↔ finger ↔ sensor slot
<code>access_events</code>	every scan: slot, score, status, reason, punch (in/out), time (UTC)

Table	Holds
<code>holidays</code> , <code>settings</code>	holiday dates; weekend days and the admin password hash
<code>sessions</code>	SHA-256 hashes of login tokens + expiry

Phone web app and Android app

Admins use one web app, which can be opened in a browser, installed as a PWA or run inside the Android app. The Android app exists because Android's WebView has no Web Bluetooth: it supplies BLE natively so the same `app.js` can enroll fingers.

Web app (`phone/`)

File	Purpose
<code>index.html</code> (441 lines)	Five tabs: Dashboard, Employees, Enroll, Attendance (weekends, holidays, reports, automatic reports), Logs
<code>app.js</code> (1,655 lines)	All UI logic: login, API calls, live updates via <code>/api/v1/changes</code> , BLE enrollment
<code>styles.css</code>	App styling
<code>sheet.html</code> / <code>.js</code> / <code>.css</code>	The "data sheet": a spreadsheet-style view at <code>/sheet.html</code>
<code>android-bridge.js</code>	Inside the Android app, wraps <code>window.AndroidBle</code> as <code>navigator.bluetooth</code> ; does nothing in a normal browser
<code>sw.js</code>	Service worker, cache <code>cbio-app-v18</code> ; bump the name to force clients to reload the app shell
<code>manifest.webmanifest</code> , <code>icons/</code>	PWA install metadata and icons
<code>serve.py</code>	Serves these files on port 8080 and forwards <code>/api/*</code> and <code>/health*</code> to the backend, so both share one origin (the backend has no CORS)
<code>download/employees-access.apk</code>	The APK users download

Run it locally with `python3 serve.py --port 8080 --backend http://127.0.0.1:8100` .

Android app (android/)

Item	Value
Package	com.employeesaccess.admin
Version	1.2 (versionCode 3), minSdk 26, targetSdk 34
MainActivity.java	A full-screen WebView loading <code>http://192.168.88.84:8080</code> (changeable in the app); exposes <code>AndroidApp (saveFile , changeServer)</code> to JavaScript
BleBridge.java	Native BLE scan, connect, read, write and notify, exposed to JavaScript as <code>AndroidBle</code>
Permissions	INTERNET, BLUETOOTH_SCAN, BLUETOOTH_CONNECT (+ legacy Bluetooth and location on Android 11 and older)
Signing key	<code>android/release.keystore</code> (alias <code>app</code>). Keep it: losing it means users cannot install updates over the old app

To build and publish the APK (no Gradle; uses the Android SDK build tools and Android Studio's JDK):

```
cd ~/fingerprint_system/android
./build.sh # → build/employees-access.apk
cp build/employees-access.apk ../phone/download/ # publish for download
```

Bump `VERSION_CODE` and `VERSION_NAME` in `build.sh` for each release.

Workflows

1. Terminal boot

1. The terminal starts the TFT, then BLE (advertising as `CBIO-ENTRY-001`).
2. It joins WiFi and syncs the clock over NTP (IST).
3. It sends `POST /devices/heartbeat` ; the backend records the terminal and its firmware version.
4. It checks the R307S password and template count, then waits for a finger.

2. Add an employee

1. The admin logs in to the web app (`/api/v1/auth/login`).
2. Employees tab → create with code, name and optional department, email and phone (`POST /employees`).
3. The code is stored in upper case and must be unique among current employees.

3. Enroll a fingerprint (phone → BLE → terminal → backend)

1. In the Enroll tab, the admin picks the employee and finger (1-10) and connects to the terminal over Bluetooth.
2. On first pairing, the terminal shows a 6-digit passkey on its screen, and the admin types it on the phone.
3. The phone writes `{employee_id, finger_index}` to the command characteristic; the terminal reports `queued`.
4. The terminal calls `POST /enrollment/start` (`contacting_server`); the backend checks the employee and returns a free sensor slot.
5. The employee places the finger, removes it, then places it again (`place_finger 1`, `remove_finger`, `place_finger 2`).
6. The terminal stores the template in that slot (`saving`) and calls `POST /enrollment/complete`.
7. The phone receives `done` with the slot and name.

4. Daily punch (access event)

1. An employee places a finger, and the R307S searches its slots for a match.
2. No match → the screen shows "Fingerprint not recognised". A match → the terminal sends `POST /access-events` with the slot and score.
3. The backend maps the slot to an employee. An unknown slot or inactive employee is denied.
4. Punches alternate per day: the first scan of the day is **IN**, then OUT, IN, OUT... A repeat scan within 30 seconds is ignored.
5. On OUT, the response includes worked time today, which the terminal shows along with the name.
6. If the server cannot be reached, the terminal still shows a local match ("granted") but no punch is recorded.
7. The web app updates live through the `/changes` long-poll.

5. Attendance and reports

- Each day is classed as `working`, `weekend` (default days set in Calendar) or `holiday`.
- Each employee gets a status of `in_office`, `present`, `absent` (working day, no punches) or `off`, plus first IN, last OUT and worked time (sum of IN→OUT pairs).
- An IN with no OUT on a past day is flagged `missed_punch_out`.
- Admins download CSVs on demand (Attendance tab), and `report.py` saves them automatically every night (see Pipelines).

6. Delete an employee

Deleting sets `deleted_at` rather than removing the row, so old logs and reports keep the name, and the code can be reused.

Pipelines

There is no CI server; every pipeline is a script or a cron job on `192.168.88.84`. The server clock runs on UTC, so the cron times below are converted to IST.

Pipeline	Trigger	Steps	Output
Firmware build	manual: <code>idf.py build flash</code>	CMake → xtensa GCC → flash over USB serial	firmware on the ESP32
Android build	manual: <code>android/build.sh</code>	<code>aapt2 resources → javac → d8 dex → zipalign → apksigner sign + verify</code>	<code>android/build/employees-access.apk</code> → copy to <code>phone/download/</code>
Deploy / restart	manual: <code>deploy/install.sh</code> or <code>docker compose restart</code>	start <code>ea-backend</code> (port 8100) and <code>ea-web</code> (port 8080) with <code>restart: unless-stopped</code> ; reinstall cron	running services
Web app update	edit files in <code>phone/</code>	served live from the mounted folder; bump <code>CACHE</code> in <code>sw.js</code> so phones fetch the new shell	updated app

Pipeline	Trigger	Steps	Output
Nightly backup	cron <code>30 20 * * *</code> UTC = 02:00 IST	<code>deploy/backup.sh</code> : SQLite online backup → delete copies older than 30 days	<code>backups/biometric-YYYYMMDD-HHMM.db</code> , log in <code>backups/backup.log</code>
Nightly reports	cron <code>45 18 * * *</code> UTC = 00:15 IST	<code>deploy/report.py</code> (reuses <code>server.py</code> report code): yesterday's daily CSVs; on the 1st, last month's too	<code>reports/daily/*.csv</code> , <code>reports/monthly/*.csv</code> , log in <code>reports/report.log</code>

The crontab on the server currently has both entries (checked 2026-10-06), and both containers have been up for 24 hours.

Where things are stored and how to access them

Everything lives on the server `192.168.88.84` under `/home/server/fingerprint_system`. Reach it with `ssh server@192.168.88.84`.

Data and files

What	Where	How to access
Live database	<code>backend/biometric.db</code> (SQLite)	<code>sqlite3 backend/biometric.db</code> on the server, or through the web app
Fingerprint templates	inside the R307S sensor's flash (slots 1-999); the DB stores only slot numbers	the terminal only; templates never leave the sensor
Nightly DB backups	<code>backups/biometric-YYYYMMDD-HHMM.db</code> (30 days)	<code>ls backups/</code> ; restore steps in Commands below
Manual DB backups	<code>backend/biometric-backup-*.db</code> , <code>backups/pre-ble-fix-20261005-0911.tar.gz</code>	files on the server
Daily reports	<code>reports/daily/attendance-DATE.csv</code> , <code>punch-times-DATE.csv</code>	web app → Attendance → Automatic reports, or the files
Monthly reports	<code>reports/monthly/attendance-YYYY-MM.csv</code> , <code>punch-times-YYYY-MM.csv</code>	same
Job logs	<code>backups/backup.log</code> , <code>reports/report.log</code>	<code>tail</code> on the server
Service logs	Docker container logs	<code>docker compose logs -f backend</code>

What	Where	How to access
Admin password	<code>settings</code> table, PBKDF2 hash	reset with <code>server.py --set-password</code>
WiFi credentials, server URL, device ID	<code>firmware/hello_world/main/hello_world_main.c</code>	edit and reflash
APK signing key	<code>android/release.keystore</code>	keep a private copy off the server
ESP-IDF toolchain	<code>tools/esp/esp-idf</code> (1.8 GB)	<code>. tools/esp/esp-idf/export.sh</code>

Network addresses

Service	URL	Who uses it
Web app	<code>http://192.168.88.84:8080</code>	admins on phones and browsers
Data sheet	<code>http://192.168.88.84:8080/sheet.html</code>	admins
APK download	<code>http://192.168.88.84:8080/download/employees-access.apk</code>	Android users
Terminal API	<code>http://192.168.88.84:8100</code>	the ESP32 terminal
Health check	<code>http://192.168.88.84:8100/health</code>	monitoring
Terminal over BLE	<code>CBIO-ENTRY-001</code>	phone, standing at the terminal

All of these are plain HTTP on the office LAN; they are not reachable from the internet. The old prototype containers (`cbio-api` on port 8000 and `cbio-postgres`) are separate and not part of this system.

Commands and procedures

Everyday operations (on the server)

```
cd ~/fingerprint_system/deploy
docker compose ps                # are ea-backend and ea-web running?
docker compose logs -f backend   # live log: punches, enrollments
docker compose restart           # after changing backend/ or phone/ code
docker compose down              # stop everything
./install.sh                     # start + (re)install the cron schedule
./backup.sh                      # back up the database now
TZ=Asia/Kolkata python3 report.py --date 2026-10-05  # (re)make one day's
reports
TZ=Asia/Kolkata python3 report.py --month 2026-09    # (re)make one month's
reports
python3 ../backend/server.py --set-password          # change the admin
password
curl http://127.0.0.1:8100/health                   # backend alive?
crontab -l                                           # check the nightly
jobs
```

Restore a backup

1. Stop the backend: `docker compose -f deploy/docker-compose.yml stop backend`
2. Copy the backup over the live DB: `cp backups/biometric-YYYYMMDD-HHMM.db backend/biometric.db`
3. Start it again: `docker compose -f deploy/docker-compose.yml start backend`

Change the server address

1. Firmware: edit `SERVER_BASE_URL` in `main/hello_world_main.c`, then rebuild and flash.
2. Android app: change the server from inside the app (no rebuild needed), or edit `DEFAULT_SERVER` in `MainActivity.java` and rebuild.
3. Browsers: open the new `:8080` URL.

Release a new app version

1. Edit `phone/` files and bump `CACHE` in `sw.js`.
2. For an Android release: bump `VERSION_CODE` / `VERSION_NAME`, run `android/build.sh`, and copy the APK to `phone/download/`.
3. Check the result in a browser and on a phone.

Troubleshooting

Symptom	Check
Terminal shows a match but no punch appears	Is the backend up (<code>/health</code>)? Is the terminal on WiFi? Did the boot log show the heartbeat?
"Fingerprint not recognised" for an enrolled person	Re-enroll that finger; check <code>access_events</code> for <code>unknown_sensor_slot</code>
Enrollment fails with "memory full"	All 999 slots are used; delete old employees' records
Phone cannot find the terminal over BLE	Use the Android app or Chrome (Web Bluetooth); be in range; re-pair to get a new passkey
Login locked	5 wrong passwords → wait 60 s; reset with <code>--set-password</code>
No new report files	<code>tail reports/report.log</code> ; check <code>crontab -l</code>
Web app shows an old version	Bump <code>CACHE</code> in <code>sw.js</code> , then reload

Housekeeping notes

- Most of the project is uncommitted in Git (only `README.md` is committed, on `master`). Commit the source, but leave out `*.db`, `backups/`, `reports/`, `build/`, `tools/esp/esp-idf` and `release.keystore`.
- `docs/`, `scripts/` and `test/` are empty.
- The WiFi password is hard-coded in the firmware source, so keep that file private.